

# Campaign Portal

Project handbook — business, architecture, and how every piece works

Prepared for the technical interview.

Repository: [github.com/FadyMasoud/Campaign](https://github.com/FadyMasoud/Campaign)

Live: [campaign-five-flax.vercel.app](https://campaign-five-flax.vercel.app)

14 commits · 11 migrations · 122 tests · 95,176 customers

## Contents

1. The business and the task
2. The data you were given
3. What Supabase and Vercel are, and how they bind together
4. How a request actually travels
5. How the spreadsheets got into the database
6. Every page, and what it is for
7. Roles and permissions
8. Sending a campaign, and hearing back
9. The guarantees, and how to defend them
10. How it was built, from nothing
11. Questions you will be asked, with answers
12. Final submission checklist

# 1 • The business and the task

## Who Velocity Growth are

Velocity Growth is a growth consultancy. They run lifecycle and retention marketing for consumer brands across the Middle East and Africa — the emails and text messages a company sends its own customers to bring them back.

The commercially important sentence in their brief is this one: *“Several client brands live inside one product, on one database, at the same time, and keeping them apart is something we promise when we sell it.”*

**That is the whole business problem.** They sell one piece of software to several competing clients. If Kilele Rides could ever see Karoo Coaches’ customer list, Velocity would lose both clients and probably the company. Everything in this build is subordinate to that promise.

## What the product does

Three brands’ marketing teams log in and work with their own growth data. Each brand has two people: an **owner** who can send campaigns, and an **analyst** who can only look. Six logins.

| Brand             | Market       | Customers | Campaigns | Engagement records |
|-------------------|--------------|-----------|-----------|--------------------|
| Kilele Rides      | Kenya        | 81,842    | 44        | 301,209            |
| Karoo Coaches     | South Africa | 12,406    | 19        | 69,100             |
| Marrakech Express | Morocco      | 928       | 6         | 1,030              |

Kilele is **88 times** the size of Marrakech. That ratio is deliberate on their part: it is how they find out whether you wrote something that works on a demo dataset or something that works on a real one.

## The five things the product must do

1. **Load each brand’s data** from the supplied files.
2. **Show three views and a dashboard** — customers, campaigns, and totals: how many customers, how many are contactable, signups per day over the last 30 days, and how each campaign performed.
3. **Send a campaign** — the owner picks one, sees exactly who it goes to and how many people that is, and confirms.
4. **Keep up with the provider** — delivery reports, bounces, opens and unsubscribes arrive over time and must keep the picture of who is contactable correct.
5. **Publish results** — turn one campaign’s results into a password-protected link for a client with no login.

**How they grade it.** Their words: *“UI and UX earn brownie points and are welcome, but the main thing we’re grading is the data and the guarantees around it.”* They also say they will sign in as each user directly against Supabase, not just through your app — so anything enforced only in the interface counts for nothing.

## 2 • The data you were given

---

A zip of eleven CSV files, verified against the SHA-256 in the brief. They are deliberately messy — this is a test of whether you notice.

| File                                 | Rows    | Delimiter        | Notes                    |
|--------------------------------------|---------|------------------|--------------------------|
| kilele-contacts.csv                  | 83,993  | comma            | byte order mark          |
| kilele-contacts-delta-2026-09-01.csv | 4,180   | comma            | corrections issued later |
| kilele-campaigns.csv                 | 46      | comma            |                          |
| kilele-events.csv                    | 312,000 | comma            |                          |
| karoo-contacts.csv                   | 13,042  | comma            | Title Case headings      |
| karoo-campaigns.csv                  | 19      | comma            |                          |
| karoo-events.csv                     | 74,000  | comma            |                          |
| marrakech-contacts.csv               | 957     | <b>semicolon</b> | French column names      |
| marrakech-campaigns.csv              | 6       | <b>semicolon</b> | comma decimals           |
| marrakech-events.csv                 | 940     | <b>semicolon</b> |                          |

### What a “contact”, a “campaign” and an “event” are

- **Contact** — one customer. Name, email, phone, country, city, when they signed up, whether they consented to marketing, whether they have unsubscribed or bounced.
- **Campaign** — one send that already happened. Its name, channel, spend, and the provider’s reported totals (sent, delivered, bounced, opens, clicks).
- **Event** — one thing one customer did: opened, clicked, bounced, complained, unsubscribed. This is the raw log the reported totals are supposed to summarise.

## Every trap in the data, and what was done about it

| What was wrong                                                                                                                                                                 | Scale     | Decision                                                                                        |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|-------------------------------------------------------------------------------------------------|
| Three different dialects: comma vs semicolon, a byte order mark, three spellings of every column ( <code>external_id</code> , <code>External Id</code> , <code>e_mail</code> ) | all files | Headings are snake-cased, then a four-entry alias table handles the French names                |
| Quoted fields containing commas and newlines — some rows span two lines                                                                                                        | ~20 rows  | A real CSV parser, never <code>split(',')</code>                                                |
| <code>status</code> spelled eleven ways: <code>active</code> , <code>ACTIVE</code> , <code>Active</code> , <code>active</code> , <code>unsubscribe</code> ...                  | ~90k rows | Normalised to four canonical values                                                             |
| <code>consent_marketing</code> in thirteen forms: <code>true</code> , <code>1</code> , <code>TRUE</code> , <code>yes</code> , <code>Y</code> , <code>f</code> ...              | ~95k rows | Normalised to a boolean; <b>blank becomes NO</b>                                                |
| <code>country</code> in 23 forms: <code>KE</code> , <code>KEN</code> , <code>kenya</code> , and <code>254</code> — the dialling code in the country column                     | ~3k rows  | Mapped to ISO-2; unrecognisable becomes empty, customer kept                                    |
| <b>400 rows declaring the wrong brand</b> — 312 rows in the Kilele file say KAROO, 88 in the Karoo file say KILELE                                                             | 400 rows  | <b>Refused in both brands.</b> An export for one brand has no business creating rows in another |
| A campaign in the Karoo file naming <code>KIL-0007</code> — a Kilele campaign — as its parent                                                                                  | 1 row     | Reference kept verbatim, link left empty, reported. The database refuses it structurally anyway |
| A header line repeated in the middle of a file                                                                                                                                 | 1 row     | Detected and refused                                                                            |
| Three NUL bytes inside customer names                                                                                                                                          | 3 rows    | Refused. PostgreSQL cannot store one, and editing somebody's name is not a fix                  |
| Duplicate customer references within one file                                                                                                                                  | 3,304     | Last one wins; the 368 that differ are reported individually                                    |
| 88 customers with signup dates <b>in the future</b> (up to June 2027)                                                                                                          | 88 rows   | Counted as customers, excluded from any time trend                                              |
| Phone numbers twelve digits behind a single zero — <code>025710676604</code>                                                                                                   | 21,974    | Number dropped, customer kept. A wrong number is a message to a stranger                        |
| Invalid emails: <code>john doe@vg-eval.test</code> , <code>double@@...</code> , <code>no-tld@vg-eval</code>                                                                    | 1,105     | Address dropped, never repaired. Deleting the space is a guess about who that person is         |
| 633 Moroccan results naming campaigns that are not in the export                                                                                                               | 633       | <b>Kept</b> , with no campaign attached — unsubscribes are among them                           |

**The principle behind every row above:** bad input is refused and reported, never stored in a half-corrected form. A refused row appears at `/portal/imports` with its line number, the field, and the offending value — so the marketer can fix the export without emailing you.

### 3 · What Supabase and Vercel are, and how they bind together

#### Supabase, in one paragraph

Supabase is a hosted **PostgreSQL database** with three things bolted on that you would otherwise build yourself:

- **Auth** — user accounts, passwords, Google sign-in, and session tokens (JWTs).
- **An automatic REST API** (a tool called PostgREST). Every table becomes an HTTP endpoint without you writing a single route.
- **Row Level Security** — PostgreSQL rules that decide, per row, who may see it. This is the important one.

#### Vercel, in one paragraph

Vercel hosts the Next.js application. It gives a public URL, and — critically — it runs **server-side code**. That matters because two secrets must never reach a browser: the Supabase service-role key and the messaging provider’s API key. Vercel is where those live.

#### How they bind: environment variables, and one naming rule

NEXT\_PUBLIC\_\*

everything else

compiled into the JavaScript bundle

stays in the Node.js process on Vercel

readable by anyone

never sent to a browser

Five variables are set in Vercel’s dashboard:

| Variable                      | What it is                   | Safe in a browser?         |
|-------------------------------|------------------------------|----------------------------|
| NEXT_PUBLIC_SUPABASE_URL      | Which Supabase project       | Yes                        |
| NEXT_PUBLIC_SUPABASE_ANON_KEY | The publishable key          | Yes — worthless on its own |
| SUPABASE_SERVICE_ROLE_KEY     | Bypasses every security rule | <b>Never</b>               |
| DISPATCHER_BASE_URL           | The messaging provider       | Yes                        |
| DISPATCHER_API_KEY            | Sends real messages          | <b>Never</b>               |

**Why the publishable key being public is fine.** It only identifies the project. On its own it opens nothing: every table refuses the anonymous role outright. Proof — with that key and no login, every table answers `42501 permission denied`, while a table that does not exist answers `404`. The difference proves the tables are there and the key cannot touch them.

### Three database clients, and which one you may use where

| Client                                     | Key          | Acts as              | Security rules  | Where       |
|--------------------------------------------|--------------|----------------------|-----------------|-------------|
| <code>createBrowserSupabaseClient()</code> | publishable  | the signed-in user   | <b>enforced</b> | anywhere    |
| <code>createServerSupabaseClient()</code>  | publishable  | the signed-in user   | <b>enforced</b> | server only |
| <code>createAdminSupabaseClient()</code>   | service role | nobody — full access | <b>bypassed</b> | server only |

The two server files begin with `import 'server-only'`. That package has no browser build, so if a browser component ever imports them — directly or through a chain of imports — **the build fails** rather than shipping a secret to the public. It is one line, and it is why the secrets stay secret.

**The admin client is a deliberate hole in the safety net.** It exists for the three jobs with no logged-in user: importing the files, reading provider reports, and serving the public shared report. Every query it makes filters by `brand_id` by hand, because the database will not do it for you there.

## 4 • How a request actually travels

Follow one click — a marketer opening the customer list:

1. Browser GET `https://campaign-five-flax.vercel.app/portal/contacts`  
cookie: sb-...-auth-token (HttpOnly – page JavaScript cannot read it)
2. Vercel proxy.ts runs first
  - reads the cookie, asks Supabase Auth "is this token still valid?"
  - refreshes it if it is close to expiring
  - no session -> redirect to /login
3. Vercel the page renders ON THE SERVER  
`createServerSupabaseClient()` attaches the user's token
4. Supabase PostgREST receives: GET `/rest/v1/contacts?select=...`  
Authorization: Bearer <the user's JWT>
5. Postgres RLS runs before any row is returned:  
`USING (brand_id IN (SELECT app.current_user_brand_ids()))`  
  
`auth.uid()` comes from the JWT -> `brand_members` -> their brand id
6. Postgres returns ONLY that brand's rows. The other 13,334 do not exist  
as far as this query is concerned.
7. Vercel renders HTML and streams it back.

**Notice what the application never does: it never filters by brand.** The query says *"give me the contacts"*, not *"give me the contacts where brand = mine"*. A filter in application code is a filter somebody can forget on the next page they add. The database cannot forget.

### Where the isolation rule lives — memorise this

```
supabase/migrations/20260913160000_brands_and_isolation.sql, line 104

CREATE FUNCTION app.current_user_brand_ids() RETURNS SETOF uuid
LANGUAGE sql STABLE SECURITY DEFINER SET search_path = ''
AS $$
  SELECT m.brand_id FROM public.brand_members m
  WHERE m.user_id = (SELECT auth.uid());
$$;
```

Every policy on every table is the same single line referring to it:

```
USING (brand_id IN (SELECT app.current_user_brand_ids()))
```

Three properties of that function, each of which you should be able to defend:

| Property                                            | Why                                                                                                                                                                                                                               |
|-----------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>SECURITY DEFINER</code>                       | It reads <code>brand_members</code> , which is itself protected by a policy that calls this function. Without it, PostgreSQL raises <i>"infinite recursion detected in policy"</i> .                                              |
| <code>SET search_path = ''</code>                   | A security-definer function with a caller-controlled search path is a privilege-escalation hole — the caller could substitute their own <code>brand_members</code> table. The empty path forces every name to be fully qualified. |
| Returns a <i>set</i> , not a boolean taking the row | A function taking <code>brand_id</code> as an argument is re-evaluated <b>once per row</b> . This one is hoisted out and evaluated <b>once per query</b> . At 81,842 contacts that is the difference between usable and not.      |

## 5 • How the spreadsheets got into the database

One command: `npm run import:seed`. Here is what happens inside it.

```
CSV file
|
| 1. READ          strip byte order mark, sniff the delimiter,
|                  snake-case the headings, alias the French ones
v
raw rows (with line numbers kept)
|
| 2. NORMALISE     status  -> one of four values
|                  consent -> boolean (blank = NO)
|                  country -> ISO-2, or empty
|                  phone   -> +254712345678, or dropped
|                  dates   -> three formats, one flagged as ambiguous
|                  money   -> 221,09 becomes 221.09
v
| 3. VALIDATE      wrong brand?      -> REFUSE, record why
|                  no status?       -> REFUSE, record why
|                  NUL byte?        -> REFUSE, record why
|                  no email AND no phone? -> REFUSE, record why
v
| 4. DEDUPE        same customer twice in one file -> last wins
v
| 5. UPSERT        INSERT ... ON CONFLICT (brand_id, external_id) DO UPDATE
|                  2,000 rows per statement, via unnest()
v
Postgres          + a row in import_runs (counts)
                  + a row in import_issues per problem (line, field, value)
```

### Why it connects directly to Postgres, not through the REST API

This moves roughly 400,000 rows. Through the REST API, one request at a time, that would take about an hour and could leave the database half-loaded if it failed in the middle. A direct connection with batched statements does it in about three minutes.

### Why running it twice is safe — the property they will test

```
INSERT INTO contacts (...) VALUES (...)
ON CONFLICT (brand_id, external_id) DO UPDATE SET ...
```

`(brand_id, external_id)` is unique. A second run *updates* the same rows instead of inserting new ones. Proven: the whole import was run twice and the second run reported **created 0** for every file with row counts unchanged.

**Why uniqueness is per brand, not global.** The three exports number their customers from the same pool — `CT-000484` exists in more than one brand and means a different person in each. A global unique constraint would have silently merged two people into one.

## The one default in the entire importer

10,056 rows have a blank consent field. Blank becomes **NO**. This is the only place a value is assumed rather than refused, and it is deliberately the cautious direction: the absence of a recorded yes cannot become permission to contact someone. Defaulting the other way would have added ten thousand people to a send nobody agreed to.

## 6 • Every page, and what it is for

| Path                        | What it shows                                                       | Why it exists                                                                                                                            |
|-----------------------------|---------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| /login                      | Email + password, and Continue with Google                          | The brief requires both methods to work                                                                                                  |
| /no-access                  | "This account has no brand"                                         | Google authenticates anyone. Authentication is not authorisation — this is where an outsider lands, seeing no brand names and no numbers |
| /portal                     | Total customers, contactable, signups per day, campaign performance | The four figures the brief names by name                                                                                                 |
| /portal/contacts            | Searchable, filterable customer list, 50 per page                   | Must stay fast at 81,842 rows — search is 276 ms                                                                                         |
| /portal/campaigns           | Every campaign, provider figures beside event-log figures           | The two disagree; both are shown rather than one chosen                                                                                  |
| /portal/campaigns/[id]      | One campaign in full, plus Send and Share panels                    | Where an owner acts                                                                                                                      |
| /portal/campaigns/[id]/send | Recipient count, the rule behind it, five real recipients, confirm  | The count shown is the count approved                                                                                                    |
| /portal/sends/[id]          | Approved / accepted / refused, per-recipient status, timeline       | "Where your send's progress and results are recorded" — asked for by name in the submission                                              |
| /portal/imports             | Each file, what loaded, what was refused and why                    | "The marketer sees what didn't load, without emailing you"                                                                               |
| /r/[token]                  | One campaign's totals, behind a password                            | For a client with no login                                                                                                               |

### The dashboard, in detail — and the idea that carries it

Every number on every screen carries a visible chip saying where it came from. Three sources, never blended:

| Basis                             | Meaning                                                                    |
|-----------------------------------|----------------------------------------------------------------------------|
| <b>Provider-reported</b>          | What the messaging provider claimed, stored exactly and never recalculated |
| <b>Counted from the event log</b> | Counted from the raw engagement rows                                       |
| <b>Derived by this portal</b>     | Worked out from customer records, by a rule printed beside the number      |

The brief asks for this directly: *"Where two careful people could reasonably count something two ways, say on the screen which way you counted."* The chip is on the page rather than in a tooltip, because a basis that only appears on hover is one most people never see — and on a phone there is no hover at all.

## Contactable — the number worth understanding properly

|                         | Kilele        | Karoo      | Marrakech  |
|-------------------------|---------------|------------|------------|
| Total customers         | 81,842        | 12,406     | 928        |
| – no consent            | 23,802        | 6,273      | 469        |
| – unsubscribed (record) | 4,236         | 483        | 0          |
| – bounced (record)      | 2,148         | 202        | 0          |
| – removed or suppressed | 701           | 0          | 0          |
| – opted out (event log) | 14,150        | 4,880      | 149        |
| – bounced (event log)   | 620           | 379        | 70         |
| <b>= contactable</b>    | <b>36,185</b> | <b>189</b> | <b>240</b> |

**Karoo is the interesting one, and you should raise it yourself.** Its customer records claim 483 unsubscribes. The event log holds **4,880 more people** who actually opted out without their record ever being updated. The portal believes the log, because the log is what happened — and says so on screen. This is also the number to name when they ask which figure you trust least.

## 7 • Roles and permissions

**An analyst sees everything an owner sees.** That is correct, not an oversight: the brief says the analyst “can only look”, and looking is the job. The roles differ only in *actions*.

| Path                                                             | Signed out | Analyst                            | Owner           |
|------------------------------------------------------------------|------------|------------------------------------|-----------------|
| /login , /r/[token]                                              | yes        | yes                                | yes             |
| /portal , /portal/contacts , /portal/campaigns , /portal/imports | redirected | identical                          | identical       |
| /portal/campaigns/[id]                                           | redirected | identical <b>minus two buttons</b> | + Send, + Share |
| /portal/campaigns/[id]/send                                      | redirected | <b>refused</b>                     | yes             |
| /portal/sends/[id]                                               | redirected | yes — may also fetch reports       | yes             |

### Only two actions are owner-only

Sending a campaign, and publishing a shared link. Both are enforced in the database, not in the interface:

```
CREATE POLICY campaign_sends_owner_may_approve ON campaign_sends
FOR INSERT TO authenticated
WITH CHECK (
  brand_id IN (SELECT app.current_user_brand_ids()) -- your brand
  AND app.is_brand_owner(brand_id) -- and you are its owner
);
```

Tested against the live deployment, signed in as the Karoo analyst, going straight at the database:

| Attempt                                 | Result                  |
|-----------------------------------------|-------------------------|
| INSERT campaign_sends (send a campaign) | <b>REFUSED — 42501</b>  |
| INSERT shared_reports (publish results) | <b>REFUSED — 42501</b>  |
| UPDATE contacts                         | <b>REFUSED — 42501</b>  |
| SELECT contacts                         | allowed — analysts read |

42501 is PostgreSQL’s *insufficient privilege*. Those attempts bypassed the application entirely — publishable key, analyst’s own session, straight to the database. That is exactly how it will be tested.

**Say this in the interview:** “Hiding the button is a courtesy so nobody clicks something they cannot do. The guarantee is the insert policy. Those are the only two tables a signed-in user may write to, and neither grants update or delete — which is what keeps an approval immutable.”

## 8 · Sending a campaign, and hearing back

### The send, in order

- |                            |                                                                                                     |
|----------------------------|-----------------------------------------------------------------------------------------------------|
| 1. requireOwner()          | the button being visible proves nothing                                                             |
| 2. re-count the audience   | if it moved since the preview: STOP, show the new number                                            |
| 3. INSERT campaign_sends   | AS THE USER, so the database's owner check runs<br>a unique index allows ONE live send per campaign |
| 4. freeze the audience     | write every recipient into send_recipients                                                          |
| 5. hand it to the provider | Idempotency-Key = this send's own id                                                                |

| Guarantee the brief asks for               | What enforces it                                                                     |
|--------------------------------------------|--------------------------------------------------------------------------------------|
| The confirmation count is what is approved | Re-counted at confirm; if it moved, nothing is sent                                  |
| No double-send                             | A partial unique index — <b>not</b> a disabled button                                |
| No silent half-send                        | Accepted and refused counted separately; any gap stated on screen                    |
| Past approvals still read as approved      | Users may <code>INSERT</code> only — no <code>UPDATE</code> , no <code>DELETE</code> |

**Why a disabled button is not enough** — they say they will press confirm “from two sessions at once”. A disabled button defends one browser tab against itself. Two sessions each have their own button. The unique index is in the one place both requests must pass through. The test fires two approvals concurrently: one wins, one is refused.

### The provider talks back

There is no webhook. The provider offers `GET /v1/messages/{batch}/events?since=`, so the app polls it — reports pile up while nothing is watching and are collected when it next looks.

**The provider’s documentation is wrong, and you should say so.** It claims: “every event is delivered exactly once and in order.” Neither half is true, and both were measured:

- Polling one batch three times returned **the same event ids all three times**.
- Timestamps within a page are not ordered — a real page reads 11:55:45, 11:56:40, 11:55:46.

The brief warned the reports would be “deliberately messy and out of order”. Where documentation and observed behaviour disagree, this build follows what it observed.

| Property                       | Mechanism                                                                                                                                |
|--------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Idempotent</b>              | Reports stored against the provider’s own <code>event_id</code> , under the same unique constraint that made re-importing a file a no-op |
| <b>Order-independent</b>       | An opt-out keeps its <i>earliest</i> timestamp, so a late report cannot undo one already recorded                                        |
| <b>Ranked, not overwritten</b> | <code>bounced</code> beats <code>delivered</code> , so a stale delivery report cannot resurrect a dead address                           |

Proven on the real send, twice in a row:

|                     | First sync | Second sync |
|---------------------|------------|-------------|
| Events read         | 320        | 20          |
| New engagement rows | 90         | 0           |
| Recipients updated  | 240        | 0           |

## The forged event — your best story

The first sync reported one unmatched row. Chasing it found this in the stream:

```
{"event_id": "evt-batch_50-forged",  
  "recipient_id": "CT-033857",  
  "brand_code": "KAR00",  
  "type": "delivered"}
```

It claims to be **Karoo**. It names a customer who belongs to **Kilele**. It arrived inside a **Marrakech** send's report stream. Three brands in one event and none of them agree. Its own id says `forged`.

It was discarded and nothing was written, because of two decisions taken before it was ever seen:

1. The provider's `brand_code` is **never read**. It returns `"account"` regardless of what was sent.
2. `recipient_id` is resolved only against customers of **the brand whose send is being synced**, never globally.

Either shortcut would have written another tenant's row. It is now a test.

## 9 • The guarantees, and how to defend them

### The isolation test, and why a passing test proves nothing

A test suite that returns empty for every query also passes. So the guarantee was deliberately broken: the `contacts` policy was replaced with `USING (true)` and the suite re-run.

```
x cannot read another brand contact, even asked for by id
x does not see another brand contact in an unfiltered listing
x is symmetric – the other brand cannot see the first either
x cannot count another brand rows
x a signed-in user belonging to no brand sees no contacts
/ sees their own contact                                <-- stayed GREEN
Tests 5 failed | 10 passed
```

The green line is the one that matters: *sees their own contact* still passed, so the suite distinguishes **isolated** from merely **empty**. The policy was then restored and every predicate re-read from `pg_policy` to confirm it matched the migration exactly.

### Isolation on tables that did not exist yet

The brief says isolation must hold “on every route into the data, including the ones added after you’ve moved on”. Two mechanisms, neither relying on memory:

1. Default privileges in `public` are **revoked** from the API roles, so a table added later is unreachable until granted deliberately — the mistake fails closed instead of open.
2. `security_coverage()` reports the posture of *every* table, and the test asserts against that live list rather than a hand-written one.

It worked twice without being edited: adding `campaign_sends` in phase 5 and `shared_reports` in phase 7 both turned the suite red immediately.

### Performance — requirement 5, measured not asserted

| Query                                        | First attempt | Now           |
|----------------------------------------------|---------------|---------------|
| Contactability (95k customers × 371k events) | 4,274 ms      | <b>299 ms</b> |
| Campaign performance (69 campaigns)          | 3,384 ms      | <b>82 ms</b>  |
| Search across 81,842 customers               | —             | 276 ms        |

Neither fix was caching:

- **Contactability** — “has this person opted out?” is a *fact*, not a question to re-ask on every page load. It became two columns on `contacts`, maintained by a trigger as events arrive.
- **Campaign performance** — the list stopped counting *distinct people* across every campaign at once. A list does not need that; only the detail page does.

## The shared link — two different problems

| Requirement                                           | Answer                                                                                                                                                                                  |
|-------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Nothing reachable by <b>guessing the address</b>      | 256-bit token from a cryptographic source (not a UUID, which carries 122 bits and a recognisable shape). An unknown token, a withdrawn one, and a wrong password all answer identically |
| Nothing reachable by <b>getting past the password</b> | There is nothing else behind it. One token resolves to one report to one campaign, and the page takes no identifier a reader could alter                                                |

The password is bcrypt at work factor 12, hashed and compared *inside the database*. The function the public page calls returns only `ok` / `denied` / `locked` — never the report — so although anyone may call it, it cannot be used as a data endpoint. Five wrong answers lock it for fifteen minutes.

## 10 • How it was built, from nothing

Nine phases, each a real commit. This order is defensible and worth explaining: the guarantees came first, the interface last.

| Phase | What                                                         | Why then                                                        |
|-------|--------------------------------------------------------------|-----------------------------------------------------------------|
| 0     | Next.js + Supabase project, environment split, three clients | Establish the security model before there is anything to secure |
| 1     | Schema, Row Level Security, the isolation test               | The guarantee is the product. Everything else sits on it        |
| 2     | Six logins, Google, roles                                    | Nothing can be tested per-user until users exist                |
| 3     | The importer and its failure report                          | No data, no dashboard                                           |
| 4     | Dashboard, customers, campaigns                              | Reading before writing                                          |
| 5     | Sending                                                      | The first thing that costs money                                |
| 6     | Provider reports                                             | Needs a real send to report on                                  |
| 7     | The shared link                                              | Needs results worth sharing                                     |
| 8     | Design, sidebar, palette, deployment                         | Last, because it is the least load-bearing                      |

### If you rebuilt it tomorrow, the order would be

1. `npx create-next-app`, then a Supabase project.
2. Split environment variables into public and server-only; add `server-only` to anything holding a secret.
3. **Write the schema and the RLS policies before any page.** One predicate function, one policy line per table.
4. **Write the isolation test, then break the policy on purpose to watch it fail.**
5. Create users; map each to a brand and role.
6. **Count the data before parsing it.** Every assumption about the files was wrong in some way.
7. Import: normalise, refuse, report, upsert.
8. Read UI, with every number labelled by its source.
9. Write paths, owner-gated in the database.
10. Design, then deploy.

**The one habit that produced most of the good decisions:** measure before deciding. The enum vocabularies, the contrast ratios, the query timings, the provider's actual behaviour, the forged event — every one of those came from running something, not from reasoning about it.

## 11 • Questions you will be asked, with answers

---

### Where does the data isolation live?

`supabase/migrations/20260913160000_brands_and_isolation.sql:104` — the function `app.current_user_brand_ids()`. Every policy in the project is the same one line referring to it. Ten tables, ten identical policies. There is no second place to look.

### What if I sign in as the Karoo analyst and query Kilele directly, bypassing your app?

You get nothing. The app is not in the loop — Row Level Security runs inside PostgreSQL, so `curl`, `psql` and the Supabase dashboard are all subject to it. Try it; that is the test I would run first too.

### How do I know your test actually tests anything?

Because I broke it on purpose. I replaced the contacts policy with `USING (true)` and five assertions failed while “sees their own contact” stayed green — which is what distinguishes isolated from empty.

### Why is the analyst seeing the same pages as the owner?

Because an analyst’s job is to look. The brief says they “can only look”, not that they see less. The difference is two actions — sending and publishing — and both are refused by the database, not by the interface.

### Your dashboard shows two different numbers for opens. Which is right?

Neither, and that is the point. The provider reports what it observed across the whole send; the event log is the engagement we hold rows for. For KAR-0001 that is 2,732 against 704. Averaging them or picking one would produce a number nobody can defend, so both are shown and labelled.

### Which number are you least sure about?

Karoo’s 189 contactable. It is correct under the stated rule, but Karoo’s records claim 483 unsubscribes while the log holds 4,880 more people who opted out. One of those sources is badly stale. I would want to know which before anyone sent to that list.

### What stops a double-send?

A partial unique index allowing one live send per campaign. Not a disabled button — that defends one browser tab against itself. Two sessions confirming at the same instant both reach the database and exactly one wins.

### What happens if your send crashes half way?

The approval and the frozen audience are already written, so the record of what was approved survives and the send is resumable. Retrying uses the same idempotency key, so the provider re-attaches to the original batch rather than sending to everyone twice.

### The provider says its reports arrive exactly once and in order. Why did you not trust that?

Because I checked. Polling one batch three times returned the same event ids three times, and timestamps within a page are not ordered. The brief also warned the reports would be messy. Where documentation and behaviour disagree, behaviour wins.

### What was the hardest bug?

A phone-number length check that accepted 8 to 15 digits turned `025701347763` into a confident `+25425701347763` — nobody's number. I replaced it with a per-market numbering plan, which then rejected 6,493 Karoo customers holding valid Kenyan +254 numbers. A number that states its own country code is unambiguous wherever it appears.

### What is not finished?

Bilingual Arabic was dropped deliberately — not in the brief, and a half-translated interface is worse than none. Delivery reports are polled on demand rather than on a schedule. And campaign pages return the not-found UI under HTTP 200, which is documented Next.js behaviour for streamed responses, mitigated by a `noindex` tag the framework injects itself.

## 12 • Final submission checklist

---

Everything the brief asks to be emailed, and where it is.

|       | Item                                | Answer                                                                |
|-------|-------------------------------------|-----------------------------------------------------------------------|
| done  | Live URL                            | campaign-five-flax.vercel.app                                         |
| done  | Six logins with passwords           | Table in README, and below                                            |
| check | Google sign-in is live              | Working — confirm once more after the Supabase URL settings are saved |
| done  | Supabase project URL + anon key     | In the note                                                           |
| done  | Table and function names            | 10 tables, 15 functions — listed in the note                          |
| done  | Which key the deployed app uses     | The publishable key everywhere user-facing                            |
| done  | Public repo with real history       | 14 commits, one per phase                                             |
| done  | schema.sql                          | Repo root, generated from 11 migrations                               |
| done  | README                              | Repo root                                                             |
| done  | AI tools used                       | docs/AI-USAGE.md — phase by phase                                     |
| done  | Where send progress is recorded     | /portal/sends/{id}, campaign_sends, send_recipients                   |
| done  | Provider key                        | The key from the brief email                                          |
| to do | Shared link + its password          | <b>Create one from any campaign page</b> — the password shows once    |
| done  | 300-word note                       | Drafted at 287 words in docs/SUBMISSION-NOTE.md                       |
| to do | How long it took                    | <b>Only you know</b>                                                  |
| to do | Earliest start date + notice period | <b>Only you know</b>                                                  |

## The eleven “done” criteria

|     | Criterion                                                                                           |
|-----|-----------------------------------------------------------------------------------------------------|
| yes | 1 · Three brands live, six logins, each landing in its own portal, whichever way they sign in       |
| yes | 2 · A brand sees its own data and nothing from another — on every route, including ones added later |
| yes | 3 · Data loads; the marketer sees what didn’t and why; loading twice leaves one set                 |
| yes | 4 · The numbers are right, and the screen says which way they were counted                          |
| yes | 5 · As usable for the 90× brand as the small one                                                    |
| yes | 6 · Sending is safe and honest — no double-send, no silent half-send                                |
| yes | 7 · The provider talks back over time, out of order, and contactability stays correct               |
| yes | 8 · A test that fails if brand isolation is removed                                                 |
| yes | 9 · The shared link is safe to send to a stranger                                                   |
| yes | 10 · Bad input rejected not stored; loading, empty and broken screens say so; AI tools named        |
| yes | 11 · A real web app — phone and laptop, client-ready                                                |

## The six logins

| Email                          | Password               | Brand             | Role    |
|--------------------------------|------------------------|-------------------|---------|
| owner@kilele.vg-eval.test      | Kilele-Owner-2026      | Kilele Rides      | owner   |
| analyst@kilele.vg-eval.test    | Kilele-Analyst-2026    | Kilele Rides      | analyst |
| owner@karoo.vg-eval.test       | Karoo-Owner-2026       | Karoo Coaches     | owner   |
| analyst@karoo.vg-eval.test     | Karoo-Analyst-2026     | Karoo Coaches     | analyst |
| owner@marrakech.vg-eval.test   | Marrakech-Owner-2026   | Marrakech Express | owner   |
| analyst@marrakech.vg-eval.test | Marrakech-Analyst-2026 | Marrakech Express | analyst |

## Before you press send

1. Save the Supabase **Site URL** and **Redirect URLs**, then try Google once — it should land on `/no-access`.
2. Publish a shared report and note the link and password.
3. Consider rotating the service-role and dispatcher keys.
4. Fill in the three bracketed facts in `docs/SUBMISSION-NOTE.md`.

**One last thing worth saying in the interview.** When they ask what you would do next: the delivery-report sync runs on demand rather than on a schedule. In production that would be a cron job hitting the same function. It is a deliberate scope decision, not something forgotten — and knowing the difference is most of what they are hiring for.